Guía de estudio — Árboles

Teoría de Grafos y Algoritmos · IPS · Repaso para el examen

CÓMO USAR ESTA GUÍA

Cubre toda la teoría de la guía “Árboles” y los ejercicios 1 a 12 de la práctica “Árboles con raíz”, resueltos con el razonamiento completo para que puedas rehacerlos solo. **Huffman no está** (no entra en el examen). Orden sugerido: teoría 1 a 7 en orden (cada tema usa el anterior) y después los ejercicios.

DOS AVISOS

1) En la guía, el algoritmo de **Prim** dice “Para i = 1 hasta m−1”. Está mal: un árbol de expansión tiene **n−1** aristas (n = vértices), así que el ciclo va hasta n−1.
2) El árbol del ejercicio 7 lo leí del dibujo del PDF. Si alguna arista no coincide con tu hoja, avisame y lo rehago.

Tema	Qué tenés que saber hacer	Dónde
Árbol y equivalencias	decidir si un grafo es árbol; contar vértices, aristas y grados	Teoría 1
Terminología con raíz	nivel, altura, padre, hijos, ancestros, hojas, internos	Teoría 2
Árbol de expansión	construirlo con DFS y BFS	Teoría 3
Árbol de expansión mínimo	Prim y Kruskal paso a paso, costo, desempates	Teoría 4
Binarios y m-arios	n = m·i + 1, ℓ = (m−1)·i + 1, altura, equilibrio	Teoría 5
ABB y recorridos	preorden, inorden, postorden, reconstrucción	Teoría 6
Expresiones y decisión	notación polaca, evaluar con pila, árboles de decisión	Teoría 7

Teoría 1 — Qué es un árbol

Un árbol es un grafo **conexo y sin ciclos**. Dicho de otra forma: entre dos vértices cualesquiera hay **un único camino**. “Único” esconde dos condiciones: existe camino (conexo) y no hay dos caminos distintos (si hubiera un ciclo, dos de sus vértices quedarían unidos por dos caminos, uno por cada lado).

TEOREMA DE EQUIVALENCIAS

Para T = (V, E) son equivalentes: (1) T es árbol. (2) T es conexo y acíclico. (3) T es simple, conexo y |E| = |V| − 1. (4) T es acíclico y |E| = |V| − 1.

Cómo se usa: para probar que algo es árbol alcanza con 2 de las 3 propiedades (conexo, acíclico, |E| = |V|−1). Pero mirar solo la cantidad de aristas no alcanza: un triángulo más un vértice suelto tiene 4 vértices y 3 aristas y no es árbol (no es conexo).

La fórmula |E| = |V| − 1 sale de construir el árbol: arrancás con 1 vértice y cada vértice nuevo entra con **una sola arista**.

Pregunta de la guía (5 nodos; 6, 5, 4 o 3 aristas): solo con **4**. Con 5 o 6 habría ciclos; con 3 no sería conexo.

Propiedades que se usan en los ejercicios

- **Suma de grados** = 2|E| = 2(|V|−1): cada arista aporta 1 al grado de cada extremo.

- **Todo árbol con n ≥ 2 tiene al menos 2 hojas** (vértices de grado 1). Si hubiera a lo sumo una, los otros n−1 tendrían grado ≥ 2 y la suma sería ≥ 2n−1 > 2n−2: contradicción.
- Agregar una arista a un árbol crea exactamente un ciclo; quitarle una lo desconecta en dos pedazos.
- **Bosque**: grafo sin ciclos. Con n vértices y c componentes tiene n−c aristas.

Aplicaciones: la llave de un torneo y un organigrama son árboles (el organigrama, con raíz).

Teoría 2 — Árboles con raíz: terminología

Un árbol con raíz es un par (T, r) con r un vértice elegido. Se dibuja con la raíz arriba. Ejemplo para fijar ideas: r tiene hijos a y b; a tiene hijos c y d; b tiene hijo e.

Término	Significado	En el ejemplo
Nivel de u	longitud (en aristas) del camino u–r; la raíz está en el nivel 0	r: 0 · a, b: 1 · c, d, e: 2
Altura	máximo nivel de sus vértices	2
Padre / hijo	el vértice anterior / siguiente en el camino desde r	padre de c = a
Ancestros de y	vértices del camino r–y, sin contar y	ancestros de c: a, r
Descendientes	w es descendiente de y si y es ancestro de w	de a: c, d
Hermanos	mismo padre	c y d
Hoja / interno	sin hijos / con hijos	hojas: c, d, e
Subárbol de z	z y todos sus descendientes	subárbol de a = {a, c, d}

Definición formal: árbol dirigido con un único vértice r de grado de entrada 0 y todos los demás con grado de entrada 1 (cada uno recibe una sola flecha, la de su padre). El **grado de salida** gs(v) es la cantidad de hijos.

> TRAMPAS
>
> El nivel cuenta **aristas**, no vértices (un árbol de un vértice tiene altura 0). Los ancestros y descendientes de y no incluyen a y. En un árbol con raíz, la raíz no es hoja aunque tenga un solo hijo.

Teoría 3 — Árbol de expansión: DFS y BFS

Un **árbol de expansión** (generador) de G es un subgrafo que es árbol y contiene todos los vértices (V' = V, E' ⊆ E). **G tiene árbol de expansión ⇔ G es conexo.** Todo árbol de expansión tiene |V|−1 aristas, y un grafo suele tener varios.

Entrada: grafo conexo y un orden de vértices v1, ..., vn (si no lo dan, el alfabético). Salida: árbol de expansión con raíz v1. Cada vez que llegás por primera vez a un vértice, la arista por la que llegaste entra al árbol.

DFS — búsqueda en profundidad (con retroceso). Usa pila o recursión

1. Parado en w, mirás los vecinos que todavía no están en el árbol. Si hay, elegís el de **menor posición** en el orden, agregás la arista y te parás en él.
2. Repetís hasta llegar a un vértice sin vecinos nuevos.
3. Retrocedés al padre y volvés al paso 1. Si el vértice sin vecinos nuevos es la raíz, termina.

Al escribir “el orden de las aristas” se listan solo los avances; los retrocesos no agregan aristas.

BFS — búsqueda en anchura. Usa cola

Primero todos los vecinos de la raíz (nivel 1), después los vecinos nuevos de esos (nivel 2), y así. Dentro de un nivel, de izquierda a derecha; para cada vértice x se agregan sus vecinos todavía no visitados, de menor a mayor posición.

Ejemplo del apunte (raíz a, orden a,b,c,d,e,f; aristas ab, ac, bd, cd, cf, de, ef)

- **DFS:** a→b→d (de d, el menor no visitado es c)→c→f→e. Aristas: ab, bd, dc, cf, fe. Después retrocede hasta a.
- **BFS:** nivel 1 = {b, c}; nivel 2 = {d, f}; nivel 3 = {e} (e cuelga de d porque d va antes que f). Aristas: ab, ac, bd, cf, de.

	DFS	BFS
Estructura	pila o recursión	cola
Cómo avanza	por una rama hasta el final, después retrocede	por niveles
Forma típica	largo y delgado	ancho y bajo
Caminos mínimos desde la raíz	no	**sí**: el nivel de cada vértice es su distancia mínima

Conexidad: con cualquiera de los dos, G (n vértices) es conexo ⇔ el árbol obtenido tiene n vértices.

Teoría 4 — Árbol de expansión mínimo (AEM)

Grafo con costos $c_e > 0$. Se busca el árbol de expansión de **menor costo total**. Tiene que ser árbol porque debe cubrir todos los vértices, ser conexo, y si tuviera un ciclo se podría sacar una arista del ciclo, seguir conectado y gastar menos.

Prim (crece desde un vértice)

1. Arrancás en el vértice inicial (el primero del orden).
2. Escribí las candidatas: aristas con un extremo adentro del árbol y el otro afuera.
3. Elegí la de menor peso (empate: según la regla del enunciado).
4. Agregá la arista y el vértice nuevo. Repetí hasta tener **n−1** aristas. Sumá los pesos.

Kruskal (arranca con todos los vértices y sin aristas)

1. Ordená todas las aristas de menor a mayor peso.
2. Recorré la lista: si los dos extremos ya están conectados por aristas elegidas, forma ciclo y se descarta; si no, se agrega.
3. Pará con n−1 aristas.

Ejemplo: las 6 ciudades (ab 4, ac 2, ae 3, bd 4, cd 1, ce 6, cf 3, df 6, ef 2)

Kruskal: orden cd(1), ac(2), ef(2), ae(3), cf(3), ab(4), bd(4), ce(6), df(6). cd ✔, ac ✔, ef ✔, ae ✔ (une {a,c,d} con {e,f}), cf ✘ (cerraría el ciclo c-a-e-f-c), ab ✔. Total 1+2+2+3+4 = **12** (5 aristas).

Prim desde a: ac(2) → cd(1) → ae(3) → ef(2) → ab(4). Total **12**.

	Prim	Kruskal
Arranca con	un vértice	todos los vértices, sin aristas
Estado intermedio	un árbol conexo que crece	un bosque que se fusiona
Qué elige	la más barata que sale del árbol	la más barata de todas que no forma ciclo
Chequeo	el otro extremo está afuera	extremos en componentes distintas

	Prim	Kruskal
Vértice inicial	sí	no

PARA RECORDAR

Goloso (greedy): en cada paso toma lo mejor disponible y nunca se arrepiente. No siempre da el óptimo, pero para el AEM **sí**. Con pesos repetidos puede haber varios AEM, todos con el **mismo costo**; si todos los pesos son distintos, el AEM es único.

Teoría 5 — Binarios, m-arios y equilibrio

Nombre	Condición	En palabras
Binario	gs(v) = 0, 1 ó 2	a lo sumo 2 hijos, distinguidos izquierdo/derecho
Binario completo	gs(v) = 0 ó 2	cada vértice tiene 2 hijos o ninguno
m-ario	gs(v) ≤ m	a lo sumo m hijos
m-ario completo	gs(v) = 0 ó m	cada interno tiene exactamente m hijos
Equilibrado	hojas en nivel h−1 ó h	h = altura del árbol

TEOREMA DEL m-ARIO COMPLETO (n vértices, i internos, □ hojas)

$n = m \cdot i + 1 \quad \ell = (m-1) \cdot i + 1 \quad i = (\ell-1)/(m-1) = (n-1)/m$

De dónde salen (para reconstruirlas sin memorizar):

1. Todo vértice menos la raíz es hijo de exactamente un interno, y cada interno tiene m hijos: n − 1 = m·i, o sea n = m·i + 1.
2. Cada vértice es hoja o interno: n = i + ℓ. Reemplazando, ℓ = (m·i + 1) − i = (m−1)·i + 1.
3. Despejando i de ambas: i = (ℓ−1)/(m−1) = (n−1)/m.

Para decidir si un árbol existe: n−1 tiene que ser múltiplo de m, y ℓ−1 múltiplo de m−1. Con m = 2: n = 2i+1 siempre es **impar** y ℓ = i + 1.

Cota de altura

En el nivel k hay a lo sumo 2^k vértices, así que un binario de altura h tiene a lo sumo 2^h hojas: **$t \leq 2^h$**, es decir **$h \geq \log_2(t)$**. Esta cota sirve para justificar imposibilidades. En un binario *completo* de altura h: máximo de hojas 2^h; mínimo h+1.

Teoría 6 — ABB y recorridos

Árbol binario de búsqueda (ABB)

Cada nodo guarda un dato; todo dato del subárbol izquierdo es **menor** y todo dato del derecho es **mayor**.

- **Insertar:** desde la raíz, si es menor vas a la izquierda y si es mayor a la derecha, hasta hallar un lugar vacío; ahí queda como hoja.
- **Buscar:** igual; si llegás a un lugar vacío, el dato no está.
- **Costo:** proporcional a la altura (hasta h+1 comparaciones). Si los datos llegan ya ordenados, el ABB degenera en una cadena.
- El **inorden** de un ABB da los datos ordenados de menor a mayor.

```
Insertando 40, 25, 60, 10, 30, 70, 65:
40
├─ (I) 25
│  ├─ (I) 10
│  └─ (D) 30
└─ (D) 60
   └─ (D) 70
      └─ (I) 65        65>40 → der · 65>60 → der · 65<70 → izq
```

Los tres recorridos (se aplican recursivamente en cada subárbol)

Recorrido	Orden	La raíz se procesa
Preorden	raíz, izquierdo, derecho	primero
Inorden	izquierdo, raíz, derecho	en el medio
Postorden	izquierdo, derecho, raíz	al final

Un subárbol vacío no aporta nada: un nodo sin hijo izquierdo simplemente salta ese paso.

Reconstruir un árbol a partir de dos recorridos

- La raíz es el **primero** del preorden o el **último** del postorden.
- El **inorden** separa: lo que está antes de la raíz es el subárbol izquierdo y lo de después, el derecho.
- Contando los vértices del lado izquierdo sabés qué tramo del pre/postorden le corresponde. Se repite en cada subárbol.
- Preorden+inorden o postorden+inorden determinan un único árbol. Un solo recorrido, en general, no alcanza.

Teoría 7 — Expresiones aritméticas y árboles de decisión

Las hojas son los operandos y los nodos internos son operadores; cada operador actúa sobre su subárbol izquierdo y derecho. El operador que se aplica **último** queda en la raíz. Importa izquierda/derecha ($4-6 \neq 6-4$).

Recorrido	Forma	Para (C + A) * E − B / D
Inorden (con paréntesis)	infija	(((C + A) * E) − (B / D))
Preorden	prefija (polaca)	− * + C A E / B D
Postorden	posfija (polaca inversa)	C A + E * B D / −

Prefija y posfija no necesitan paréntesis ni reglas de prioridad.

- **Evaluar posfija con pila:** de izquierda a derecha; operando → se apila; operador → se desapila y, luego x, y se apila ($x \circ y$). Al final queda un único valor.
- **Evaluar prefija:** recursivamente, un operador va seguido de dos expresiones completas que son sus operandos.
- Ejemplo posfija C A + E * B D / − con A=2, B=8, C=3, D=4, E=5: 3 2 + → 5; 5·5 → 25; 8 4 / → 2; 25 − 2 = **23**.

Árboles de decisión

Cada vértice interno es una prueba (un hijo por resultado) y cada hoja es una conclusión. La **altura** es la cantidad de pruebas del peor caso. Con k pruebas de b resultados se distinguen a lo sumo b^k casos, así que para n casos hacen falta al menos $\lceil \log_b n \rceil$ pruebas. Moneda falsa con 8 monedas: $3^1 = 3 < 8$ pero $3^2 = 9 \geq 8$ → **2 pesadas** (comparando 3 contra 3).

Práctica “Árboles con raíz” — ejercicios 1 a 12

Ejercicio 1 — Graficar si es posible

(a) Árbol con todos los vértices de grado 2 → imposible. Si todos tuvieran grado 2 la suma de grados sería 2n, pero en un árbol es 2(n−1) = 2n−2. (Otra forma: todo árbol con n ≥ 2 tiene al menos 2 hojas, de grado 1. Lo único con todos de grado 2 son los ciclos.)

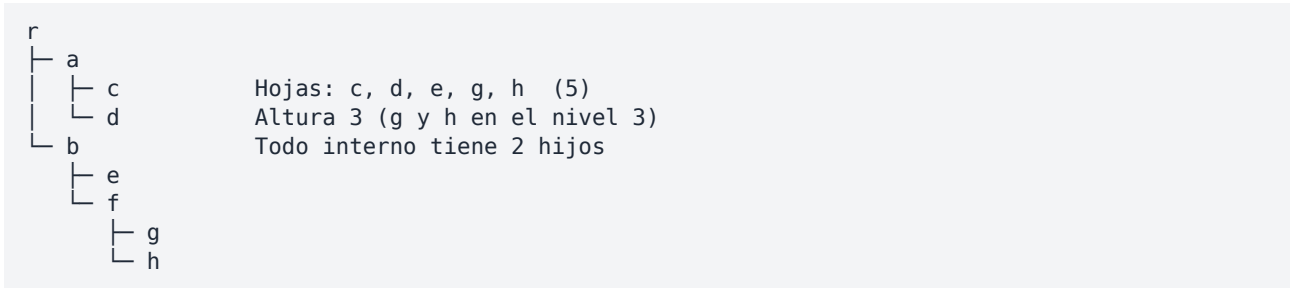
(b) 6 vértices con grados 1,1,1,1,3,3 → posible. Chequeo: suma = 10 = 2(6−1) ✔. Dos vértices de grado 3 adyacentes, cada uno con 2 hojas:



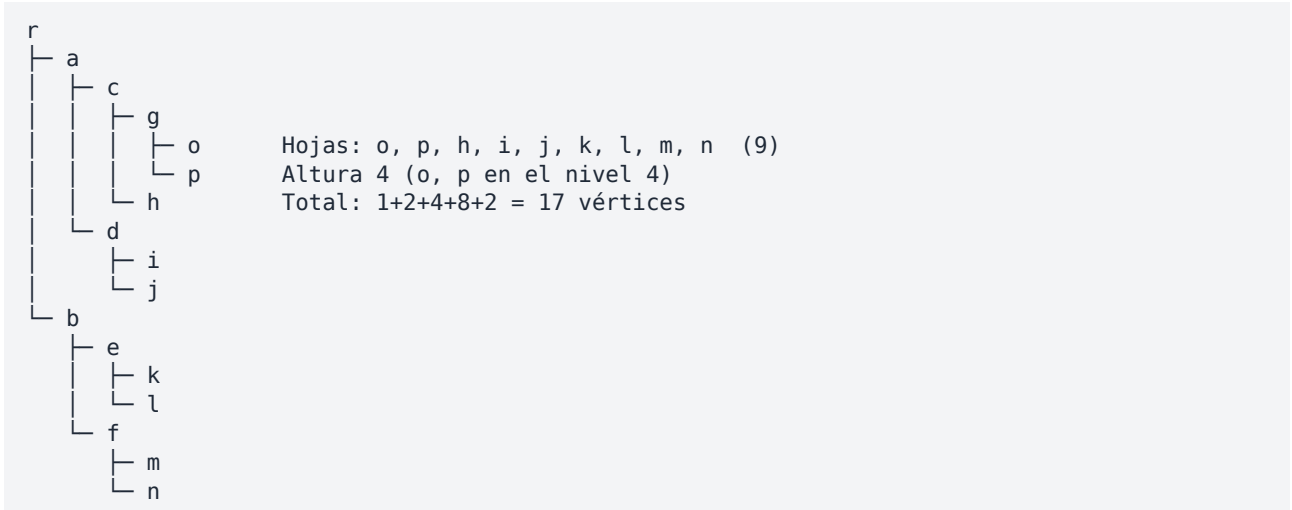
(c) 4 vértices internos y 6 terminales → posible. Acá “árbol” es general: n = 10, aristas = 9, suma de grados = 18 = 6 (hojas) + 12 (internos), así que los 4 internos pueden tener grado 3. Camino p–q–r–s con hojas: a y b en p, c en q, d en r, e y f en s. Aristas: pq, qr, rs, pa, pb, qc, rd, se, sf. (Si pidiera binario *completo*, sería imposible: ahí $\ell = i+1 = 5$.)

(d) Binario completo, altura 3, 9 hojas → imposible. Con altura 3 hay a lo sumo $2^3 = 8$ hojas, y 9 > 8 (equivalente a $h \geq \log_2 9 \approx 3{,}17$, que no se cumple con h = 3).

(e) Binario completo, altura 3, 5 hojas → posible. 5 hojas ⇒ i = 4 internos, n = 9; además $3 \geq \log_2 5$ ✔.



(f) Binario completo, altura 4, 9 hojas → posible. 9 hojas ⇒ i = 8, n = 17. Idea: partí del árbol lleno de altura 3 (8 hojas) y a *una* hoja dale dos hijos: queda 8 − 1 + 2 = 9 hojas y altura 4.



Ejercicio 2 — Máxima altura de un binario con n vértices

n − 1. Cada nivel tiene al menos 1 vértice, así que con n vértices hay a lo sumo n niveles (de 0 a n−1). Se logra con una cadena donde cada vértice tiene un único hijo.

Ejercicio 3 — Máxima altura de un binario completo con n vértices

(n − 1)/2. Como n = 2i+1, i = (n−1)/2. Si la altura es h, el camino de la raíz a la hoja más profunda tiene h vértices internos (todos menos la hoja), entonces h ≤ i = (n−1)/2. Se alcanza con un “peine”: cada interno tiene una hoja y otro interno, y el último interno tiene dos hojas.

Ejercicio 4 — Ternario completo con 34 vértices

m = 3, n = 34. De n = m·i + 1: 34 = 3i + 1 ⇒ **i = 11**. Hojas: ℓ = (m−1)·i + 1 = 2·11 + 1 = **23**. Aristas: n − 1 = **33**. Chequeo: 11 + 23 = 34 ✔.

Ejercicio 5 — 5-ario completo con 817 hojas

i = (ℓ−1)/(m−1) = 816/4 = **204 vértices internos**.

Ejercicio 6 — Las 25 computadoras

Modelo: árbol 4-ario completo. La raíz es el enchufe de pared (4 salidas), cada cable es un vértice interno (4 salidas) y las computadoras son las hojas. ℓ = 25, m = 4 ⇒ i = (25−1)/3 = 8 internos. Esos 8 incluyen al enchufe de pared, que no es cable: cables = 8 − 1 = **7**.

Chequeo directo: el enchufe da 4 salidas; cada cable ocupa una y agrega 4, o sea suma 3 netas. Con c cables: 4 + 3c ≥ 25 ⇒ c ≥ 7. Con 7: 4 + 21 = 25 exacto ✔.

Ejercicio 7 — Recorridos

Estructura leída del dibujo (I = hijo izquierdo, D = hijo derecho):

```
r
├─ (I) j
│  ├─ (I) h
│  │  └─ (D) g
│  │     ├─ (I) e
│  │     │  └─ (D) d
│  │     │     ├─ (I) b
│  │     │     │  └─ (I) a
│  │     │     └─ (D) c
│  │     └─ (D) f
│  └─ (D) i
└─ (D) k
   ├─ (I) m
   │  └─ (D) p
   │     └─ (I) s
   └─ (D) n
      └─ (D) q
         ├─ (I) t
         │  ├─ (I) v
         │  └─ (D) w
         └─ (D) u
```

Recorrido	Resultado
Preorden (raíz, izq, der)	**r j h g e d b a c f i k m p s n q t v w u**
Inorden (izq, raíz, der)	**h e a b d c g f j i r m s p k n v t w q u**
Postorden (izq, der, raíz)	**a b c d e f g h i j s p m v w t u q n k r**

Detalle para no trabarte: h no tiene hijo izquierdo, así que en el inorden va primero h y después todo su subárbol derecho (g…). Chequeo: cada lista tiene los 21 vértices; r es primero en preorden y último en postorden.

Ejercicio 8 — Listas iguales

(a) Preorden = inorden. Si hay un hijo izquierdo, en el preorden la raíz va *antes* de ese subárbol y en el inorden *después*, así que las listas difieren. Entonces nadie puede tener hijo izquierdo. Ejemplo: cadena a → b → c hacia la derecha; ambas dan **a b c**.

(b) Postorden = inorden. Si hay hijo derecho, en el inorden la raíz va *antes* de ese subárbol y en el postorden *después*. Entonces nadie puede tener hijo derecho. Ejemplo: c tiene hijo izquierdo b y b tiene hijo izquierdo a; ambas dan **a b c**.

(c) Con árboles binarios completos: no existen. Un completo con |V| ≥ 3 tiene la raíz con dos hijos, o sea subárbol izquierdo y derecho no vacíos: preorden ≠ inorden (por el izquierdo) y postorden ≠ inorden (por el derecho).

Ejercicio 9 — Reconstruir con preorden e inorden

Preorden: **G E A I B M C L D F K J H** · Inorden: **I A B E G L D C F M K H J**

1. Raíz = G (primero del preorden). En el inorden, a su izquierda {I, A, B, E} (4 vértices) y a su derecha {L, D, C, F, M, K, H, J} (8).
2. **Lado izquierdo** (preorden E A I B, inorden I A B E): raíz E, y todo lo demás está a su izquierda. Queda (preorden A I B, inorden I A B): raíz A con hijo izquierdo I e hijo derecho B.
3. **Lado derecho** (preorden M C L D F K J H, inorden L D C F M K H J): raíz M. Izquierda {L, D, C, F}, derecha {K, H, J}.
4. Izquierda de M (preorden C L D F, inorden L D C F): raíz C; a su izquierda {L, D} y a su derecha {F}. Para {L, D}: raíz L con D como hijo derecho.
5. Derecha de M (preorden K J H, inorden K H J): raíz K sin hijo izquierdo; su derecha es {H, J}: raíz J con hijo izquierdo H.

```
G
├─ (I) E
│  └─ (I) A
│     ├─ (I) I
│     └─ (D) B
└─ (D) M
   ├─ (I) C
   │  ├─ (I) L
   │  │  └─ (D) D
   │  └─ (D) F
   └─ (D) K
      └─ (D) J
         └─ (I) H
```

(b) Postorden: I B A E D L F C H J K M G (13 vértices, la raíz G queda última).

Ejercicio 10 — Binario completo con postorden dado

V = {a, ..., k} (11 vértices ⇒ i = 5, ℓ = 6). Postorden: **d e b h i f j k g c a**.

Método: el último es la raíz (a); el anterior (c) es la raíz del subárbol derecho. El resto se parte en un prefijo (subárbol izquierdo) y un sufijo (subárbol derecho). En un binario completo cada subárbol tiene cantidad impar de vértices. Dentro de cada trozo se repite: el último es su raíz.

(a) Altura 3. Parto 3 + 7: izquierda = d e b (raíz b con hijos d, e); derecha = h i f j k g c (raíz c), que se parte en h i f (raíz f, hijos h, i) y j k g (raíz g, hijos j, k).

```
a
├─ (I) b
│  ├─ (I) d
│  └─ (D) e
└─ (D) c
   ├─ (I) f
   │  ├─ (I) h
   │  └─ (D) i
   └─ (D) g
      ├─ (I) j
      └─ (D) k
```

(b) El subárbol izquierdo tiene altura 3. Un completo de altura 3 tiene al menos 7 vértices, así que el prefijo tiene 7. Parto 7 + 3: derecha = k g c (c con hijos k, g). Izquierda = d e b h i f j, raíz j: su derecha es f (hoja) y su izquierda es d e b h i (raíz i con hijo derecho h e izquierdo b(d, e)).

```
a
├─ (I) j
│  ├─ (I) i
│  │  ├─ (I) b
│  │  │  ├─ (I) d
│  │  │  └─ (D) e
│  │  └─ (D) h
│  └─ (D) f
└─ (D) c
   ├─ (I) k
   └─ (D) g
```

OJO

El postorden solo **no** determina un único árbol (no dice quiénes son hojas). Por eso el enunciado agrega la condición de altura. Pueden existir otros dibujos válidos: lo importante es que cumpla el postorden, sea completo y tenga la altura pedida.

Ejercicio 11 — (w + x − y)/(π ∗ z³) en notación polaca

La raíz es la operación que se hace última: "/". A la izquierda, w + x − y se evalúa de izquierda a derecha, o sea (w + x) − y. A la derecha, π ∗ z^3 = π ∗ (z ↑ 3).

```
/
├─ (I) −
│  ├─ (I) +
│  │  ├─ w
│  │  └─ x
│  └─ (D) y
└─ (D) *
   ├─ (I) π
   └─ (D) ↑
      ├─ z
      └─ 3
```

Preorden (raíz, izquierdo, derecho): **/ − + w x y * π ↑ z 3**

Ejercicio 12 — Evaluar / ↑ a − b c + d ∗ e f

Con a = c = d = e = 2 y b = f = 4. Se agrupa así: "/" ("↑"(a, "−"(b, c)) , "+"(d, "∗"(e, f))).

- Numerador: ↑ a (− b c) = $2^{(4-2)} = 2^2$ = **4**.
- Denominador: + d (∗ e f) = 2 + (2·4) = **10**.
- Resultado: 4/10 = **2/5 = 0,4**.

Resumen de trampas para el examen

- Árbol con n vértices ⇒ **n−1 aristas**. Chequealo primero.
- m-ario completo: **n = m·i + 1** y **ℓ = (m−1)·i + 1**.
- Altura de un binario con t hojas: **$t \leq 2^h$**. Sirve para las imposibilidades.
- Reconstruir: raíz del pre/postorden + inorden que parte izquierda y derecha.
- Kruskal: ordenás y evitás ciclos. Prim: crecés desde un vértice con la arista más barata que sale. Ambos dan el **mismo costo**. Prim hace **n−1** pasos.
- BFS = niveles = distancias mínimas. DFS = retroceso.
- En los dibujos de árboles con raíz el nivel cuenta aristas, no vértices.